

How the System Works

A plain-language tour of the platform: what a customer is, what a tenant is, what an employee is, where “the main Odoo” sits, and what all the layers above it actually do.

The whole thing in one sentence: one Odoo program serves many separate databases, and each customer company gets its own database. Everything else — nginx, the SaaS layer, the branding, the licensing — is machinery that makes that single idea *safe*, *sellable*, and *branded*.

1 · The vocabulary that trips people up

These words get used loosely everywhere. Here is exactly what each one means in *this* system:

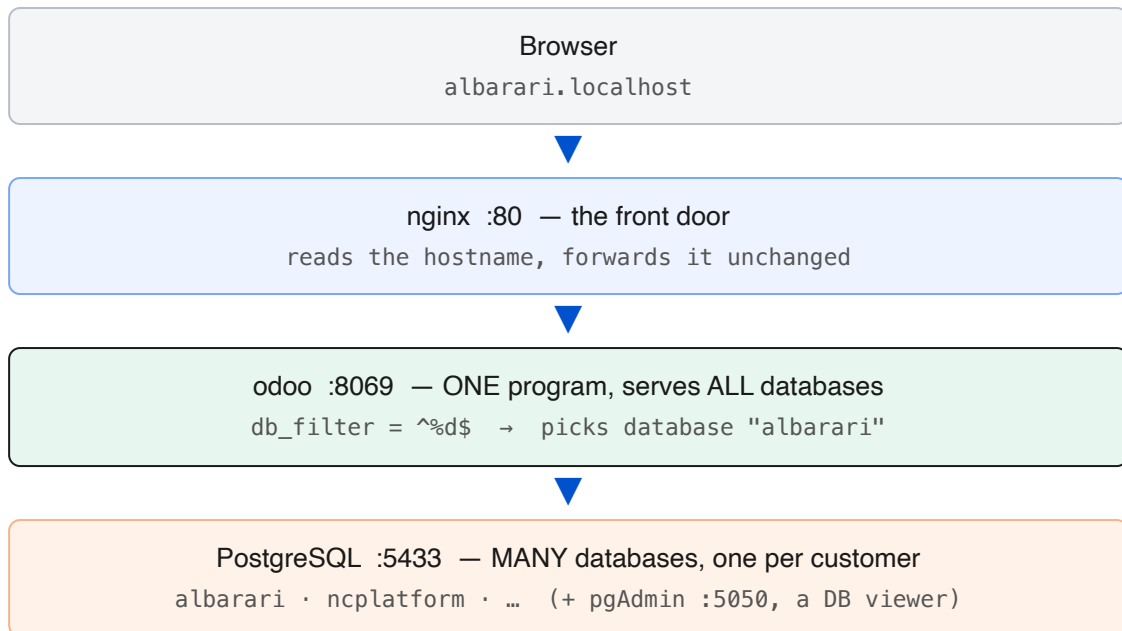
Word	What it really is	Concrete example
You / NCollection	The SaaS vendor. You own the platform and sell subscriptions.	—
Customer = Tenant	A <i>company</i> that buys a subscription. One customer = one database.	Al Barari Trading → database <code>albarari</code>
Employee = User	A <i>person</i> who works at that company — a <code>res.users</code> row <i>inside</i> that company’s database.	Fatima Rahmani → <code>fatima@albarari.ae</code>
Plan	What the customer bought. Decides which apps they may use.	DEMO plan → <code>crm</code> , <code>sale</code> , <code>account</code>
Role	What one employee may do <i>inside</i> their company.	Fatima = Accountant

Read this twice

Customer and tenant are the same thing — “customer” is the business word, “tenant” is the technical word. And a **customer is a database**, while an **employee is a row inside it**. That single distinction clears up most of the confusion.

2 · What is physically running

On the machine there are four Docker containers. The one crucial fact: **there is only one Odoo process**. It does *not* run a copy per customer — it looks at who is asking and opens the right database.



The request path. Odoo is a single engine; the database is chosen per request, not per process.

3 · The two layers — the core idea

Your databases split into two completely different *kinds*. Getting this split is the key to understanding everything else.

Platform layer — the business of selling ERP

Database: `ncplatform` — **your** admin system, not a customer's.

- `ncollection_subscription` — who are the customers, what plan, how many seats
- `ncollection_saas` — the **provisioning engine** that creates new customer databases

Tenant layer — one per customer

Databases: `albarari`, and one per future customer.

- `ncollection_core` — roles, licence enforcement, the dashboard
- `ncollection_branding` — makes it look like NCollection, not Odoo
- `crm · sale · account` — the standard Odoo apps they paid for

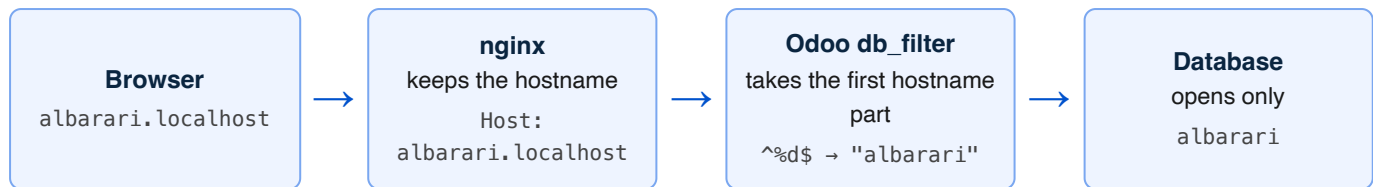
Platform layer sells and builds workspaces (left). Tenant layer is the workspace (right). The provisioning engine on the left builds each database on the right.

The hard rule (Standing Rule 3)

The platform layer must **never** reach directly into a tenant database. When `ncollection_saas` builds `albarari`, it launches a *separate Odoo subprocess* rather than reaching across. This is why one customer's data can never leak into another's, even by a programming mistake.

4 · How a request finds the right customer

You visit `albarari.localhost`. The routing is one small trick repeated reliably:



The subdomain is the database name. That is the entire routing mechanism.

Two consequences fall straight out of this:

- **Database names must be plain letters/numbers.** `db_filter` matches the hostname to the database name literally — which is why the routing fixtures had to be `rtclienta`, not `rt-clienta`.
- **Sessions are database-scoped.** Logging into `albarari` gives a cookie that is meaningless on any other tenant. `make routing-verify` proves this eight different ways.

5 · Licensing — two rings, not one

Al Barari bought `crm`, `sale`, `account`. Suppose they had *not* bought `sale`. Two independent mechanisms enforce that:

Ring 1 — cosmetic P1-T09

`ir_ui_menu.py` — the Sales menu simply does not appear in the interface.

Ring 2 — actual security P1-T10

`license_enforcement.py` — reading `sale.order` raises **AccessError** at the database layer, no matter how you ask.

Why two?

Ring 1 alone would be worthless — anyone could type a URL and reach the data. **Hiding a menu is not security.** Ring 2 is what actually stops them. You can see this live: `e2eclientb` has `sale` installed but not licensed, and reading `sale.order` returns “The ‘sale.order’ feature is not included in your *NCollection plan*.”

6 · Roles — what one employee may do

Inside every tenant there are **8 roles**: Owner, CEO, Manager, Sales, Warehouse, HR, Accountant, Employee. Each role does **two separate things**, and it matters that they are separate:

A role grants...	...which means
Odoo access (an app group)	Accountant → <code>account.group_account_user</code> → can actually open Accounting
Dashboard widgets (a widget group)	Accountant → the <code>financial</code> widget group → sees receivables, payables, cash

That is why the demo looks different per login: Fatima (Accountant) sees receivables and cash; Yousef (Sales) sees the pipeline; Aisha (Employee) sees only her own activities. The gating is enforced **server-side** — a widget a role may not see is absent from the data sent to the browser, not merely hidden.

7 • Your databases, decoded

This is the table that would have saved the “everything is broken” scare:

Database	What it is
<code>ncplatform</code>	Your SaaS control plane — where customers and plans live
<code>albarari</code>	A real customer workspace (Al Barari Trading) — the one worth looking at
<code>ncollection</code>	Your personal dev sandbox
<code>rtclienta</code> / <code>rtclientb</code> / <code>rtadmin</code>	Routing test fixtures — <code>base</code> only, empty by design
<code>e2eclienta</code> / <code>e2eclientb</code> / <code>e2eadmin</code>	Automated-test fixtures — rebuilt constantly
<code>postgres</code>	PostgreSQL’s own bookkeeping database

This is exactly what misled you

You logged into `clienta`, which was a *routing test artifact* holding nothing but Odoo’s skeleton, and reasonably concluded the product was broken. It was never a workspace. That is precisely why those fixtures were renamed to the obvious `rt*` prefix.

8 • Signup, end to end

- Someone signs up → a record is created in `ncplatform` (the platform layer).
- The **provisioning engine** creates a new database, e.g. `theircompany`.
- It installs `ncollection_core` + `ncollection_branding` + whatever apps their plan licensed.
- It writes their plan into `workspace.config` — the record that drives licence enforcement.
- They visit `theircompany.nccollectionerp.com` → `db_filter` routes them → their own isolated world.

You watched this happen for real when `albarari` was provisioned through the engine — its first real-world use outside its own tests.

9 · The mental model, compressed

Odoo is the engine. **nginx** is the doorman who reads the nameplate. **PostgreSQL** holds one sealed room per customer. **ncplatform** is your front office that sells rooms and builds them. **ncollection_core** is what makes each room *yours* rather than generic Odoo — the roles, the licensing, the dashboard. An **employee** is a person inside one room, who can never see into another.

Appendix · What the “it’s broken” investigation actually found

The platform was not broken — every tenant returned a normal page with no errors. But taking the report seriously surfaced **three genuine defects**, all invisible for the same reason: *nothing in the system had data in it*.

Defect	What / why	Status
Branding not loading	One easily-missed ERROR line. Odoo never retroactively installs a <i>newly-declared</i> dependency into an <i>existing</i> database — only an upgrade does.	Fixed + guarded (make doctor)
Cash & Bank always 0	It summed bank/cash <i>journal</i> lines, which always balance to zero. The right measure is the cash/bank <i>account</i> balance.	Fixed (R-013)
Provisioned roles grant no access	New tenants get roles that grant dashboard widgets but not the underlying Odoo app access, so an Accountant lands on an empty dashboard.	Open (#167)

The lesson worth keeping — ledger R-013

A KPI that reads 0 on empty data is not evidence that it works. All three bugs slipped past review, unit tests and CI because an empty tenant returns the same harmless-looking 0 whether the code is right or wrong. Widgets are only meaningfully verified against a *populated* tenant — which is what make demo-tenant now provides.